ExpeRepair: Dual-Memory Enhanced LLM-Based Repository-Level Program Repair

FANGWEN MU*, State Key Laboratory of Intelligent Game, Institute of Software, CAS, China
JUNJIE WANG*, State Key Laboratory of Intelligent Game, Institute of Software, CAS, China
LIN SHI, Beihang University, China
SONG WANG, York University, Canada
SHOUBIN LI*, State Key Laboratory of Intelligent Game, Institute of Software, CAS, China
QING WANG*†, State Key Laboratory of Intelligent Game, Institute of Software, CAS, China

Automatically repairing software issues remains a fundamental challenge at the intersection of software engineering and AI. Although recent advances in Large Language Models (LLMs) have demonstrated potential for repository-level repair tasks, current methods exhibit two notable limitations: (1) they often address issues in isolation, neglecting to incorporate insights from previously resolved issues, and (2) they rely on static, rigid prompting strategies that constrain their ability to generalize across diverse and evolving contexts. We propose ExpeRepair, a novel LLM-based program repair framework inspired by the dual-memory systems of human cognition, where episodic and semantic memory synergistically support learning and decision-making. Unlike existing methods, ExpeRepair continuously learns from historical repair experiences via dual-channel knowledge accumulation, enabling it to adaptively reuse past knowledge during inference. Specifically, ExpeRepair organizes prior repair knowledge into two complementary memories: an episodic memory that stores concrete repair demonstrations, and a semantic memory that encodes abstract, reflective insights. At inference time, ExpeRepair activates both memory systems by retrieving relevant demonstrations from episodic memory and recalling high-level repair insights from semantic memory. It further enhances adaptability through dynamic prompt composition, integrating both memory types to replace static prompts with context-aware, experience-driven prompts. We evaluate ExpeRepair on two benchmarks: SWE-Bench Lite and SWE-Bench Verified. Experimental results show that ExpeRepair achieves pass@1 scores of 60.3% and 74.6% on the two benchmarks, respectively, achieving the best performance among the evaluated open-source methods. We have open-sourced ExpeRepair at `https://github.com/ExpeRepair/ExpeRepair`.

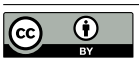
CCS Concepts: • **Software and its engineering → Software creation and management**; **Software development techniques**.

Additional Key Words and Phrases: Automated Program Repair, Large Language Model, AI Agents

ACM Reference Format:
Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. 2026. ExpeRepair: Dual-Memory Enhanced LLM-Based Repository-Level Program Repair. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE174 (July 2026), 23 pages. https://doi.org/10.1145/3808181

*Also With University of Chinese Academy of Sciences
†Corresponding author

Authors' Contact Information: Fangwen Mu, fangwen2020@iscas.ac.cn, State Key Laboratory of Intelligent Game, Institute of Software, CAS, Beijing, China; Junjie Wang, State Key Laboratory of Intelligent Game, Institute of Software, CAS, Beijing, China, junjie@iscas.ac.cn; Lin Shi, Beihang University, Beijing, China, shilin@buaa.edu.cn; Song Wang, York University, Toronto, Canada, wangsong@yorku.ca; Shoubin Li, State Key Laboratory of Intelligent Game, Institute of Software, CAS, Beijing, China, shoubin@iscas.ac.cn; Qing Wang, State Key Laboratory of Intelligent Game, Institute of Software, CAS, Beijing, China, wq@iscas.ac.cn.



ACM 2994-970X/2026/7-ARTFSE174
https://doi.org/10.1145/3808181

1 Introduction

Automated Program Repair (APR) aims to automatically identify and correct bugs in source code, offering the potential to significantly reduce developer effort and improve software reliability [23]. Traditional APR techniques have predominantly focused on function-level APR, targeting small, isolated code fragments [16]. However, this narrow scope diverges from real-world software maintenance practices, which typically require reasoning over large codebases, interpreting complex issue descriptions, and addressing inter-file dependencies. Recent advances in Large Language Models (LLMs), particularly in code generation, have shifted attention toward applying LLMs to repository-level APR [6, 46, 49, 53]. Unlike function-level APR, repository-level APR involves navigating large codebases, understanding intricate program logic, and identifying subtle bugs embedded across multiple files, introducing new and underexplored challenges.

To address these challenges, recent LLM-based APR methods fall into two main categories: agentic and procedural. Agentic approaches [1, 38, 49, 53] treat the LLM as an autonomous agent that interacts with tools, navigates repositories, modifies code, and runs tests to resolve bugs. Procedural approaches [28, 46], in contrast, follow predefined pipelines, where the LLM sequentially performs tasks such as bug localization and patch generation using fixed prompts, without autonomous decision-making. While both paradigms show promise, they exhibit two key limitations.

Neglecting Historical Repair Experience. Current approaches treat each issue in isolation and primarily rely on feedback from the ongoing resolution process, while largely ignoring valuable historical experience accumulated from previously resolved issues within the same repository. In practice, software projects often exhibit recurring bug patterns [27], such as incorrect API usage, type mismatches, or improper resource management. These patterns may manifest as syntactically similar changes or as diverse edits with similar underlying repair logic. Effectively capturing and leveraging historical repair knowledge could improve both repair efficiency and success rates.

Limited Dynamic Adaptability Due to Static Prompting. Current LLM-based repair methods rely on static, manually crafted prompts. However, these prompts fail to account for substantial variations across real-world software projects, including differences in coding conventions, architectural patterns, and project-specific practices. Manually customizing prompts per repository or issue type is impractical, particularly in large software ecosystems [20]. Moreover, static prompts cannot evolve in response to task feedback or adapt to nuanced context shifts during multi-step repair workflows. Addressing this limitation requires dynamic prompt generation or adaptation that tailors instructions based on repository characteristics, issue context, and insights from past repairs, enabling more robust, context-aware repair strategies.

In real-world software development, human programmers continuously accumulate knowledge from past debugging and repair activities, refining their strategies and reasoning over time. Inspired by this human-like learning process [25, 54], we explicitly incorporate historical repair experiences into repository-level APR, aiming to model how developers learn, adapt, and improve across issues within a software project. To effectively capture and reuse these accumulated experiences, we draw further inspiration from cognitive science. The dual-memory system theory [39, 40] posits that human reasoning and decision-making rely on two complementary types of memory: episodic memory, which stores concrete personal experiences, and semantic memory, which captures abstract, generalizable knowledge. These two systems work synergistically, enabling individuals to recall specific events while applying broader insights to novel situations.

To this end, we propose ExpeRepair, a novel LLM-based APR approach that continuously accumulates and reuses historical repair experiences through a dual-memory system. ExpeRepair comprises two core modules: a Program Repair Module and a Memory Module. The repair module orchestrates a test agent and a patch agent, which collaboratively perform test generation, patch

generation, and patch validation for each issue. The memory module organizes repair experiences into two complementary memories: episodic memory, which stores concrete repair demonstrations, and semantic memory, which distills abstract, natural language insights from past repair trajectories. At inference time, ExpeRepair retrieves relevant repair demonstrations and reflective insights to generate dynamic, context-aware prompts tailored to the current issue and repository. By maintaining both episodic memory and semantic memory, ExpeRepair achieves synergistic benefits: specific demonstrations provide precise implementation references, while reflective insights enable generalized adaptation to new and evolving contexts.

We evaluate our approach on SWE-Bench Lite and Verified [21], two challenging benchmarks consisting of real-world software issues sourced from popular GitHub repositories. Experimental results show that our approach outperforms all state-of-the-art open-source baselines, achieving pass@1 scores of 60.3% and 74.6% on SWE-Bench Lite and Verified, respectively, using Claude 4 Sonnet [4]. Furthermore, ablation studies validate the effectiveness of our dual-memory mechanism in enhancing repair performance. Our main contributions are as follows:

- We propose ExpeRepair, an LLM-based APR method that continuously learns from historical repair experiences through dual-channel knowledge accumulation.
- We design a dynamic, experience-driven prompt generation mechanism that tailors repair strategies to the specific characteristics of each issue and repository. This mechanism addresses the limitations of static, handcrafted prompts by leveraging both concrete repair demonstrations and abstract repair patterns.
- We conduct comprehensive experiments on SWE-Bench, showing state-of-the-art performance and validating the contributions of each component through ablation studies.
- We provide publicly accessible source code [13] to facilitate replication of our study and its application in broader contexts.

2 Method

As illustrated in Figure 1, ExpeRepair is built upon two key modules: the Program Repair Module and the Memory Module. The program repair module comprises a test agent and a patch agent, which collaboratively perform three essential tasks: test generation, patch generation, and patch validation. Once the program repair module processes an issue, the memory module captures the corresponding repair trajectory, extracting concrete demonstrations and summarizing high-level repair insights. These are stored in episodic memory and semantic memory, respectively. In subsequent repairs, ExpeRepair dynamically retrieves relevant demonstrations and insights to inform and enhance its repair strategies for new issues. In this section, we first present the end-to-end workflow of ExpeRepair, followed by a detailed illustration of its two core modules.

2.1 Workflow of ExpeRepair

Conventional learning approaches [17, 18, 34, 44] typically rely on computationally expensive fine-tuning to improve the capabilities of LLM-based agents. In contrast, ExpeRepair equips LLM agents with the dual-memory mechanism, enabling them to continuously acquire knowledge from their own repair experiences without requiring parameter updates. However, directly deploying ExpeRepair introduces a cold-start problem, where no historical repair experience is initially available to populate the memory systems. To overcome this, ExpeRepair adopts a two-phase strategy. In the initial phase, it is applied to a seed set of software issues to generate corresponding repair trajectories. The Memory Module then organizes these trajectories into two complementary memories. During this phase, ExpeRepair operates without accessing prior memories, and its

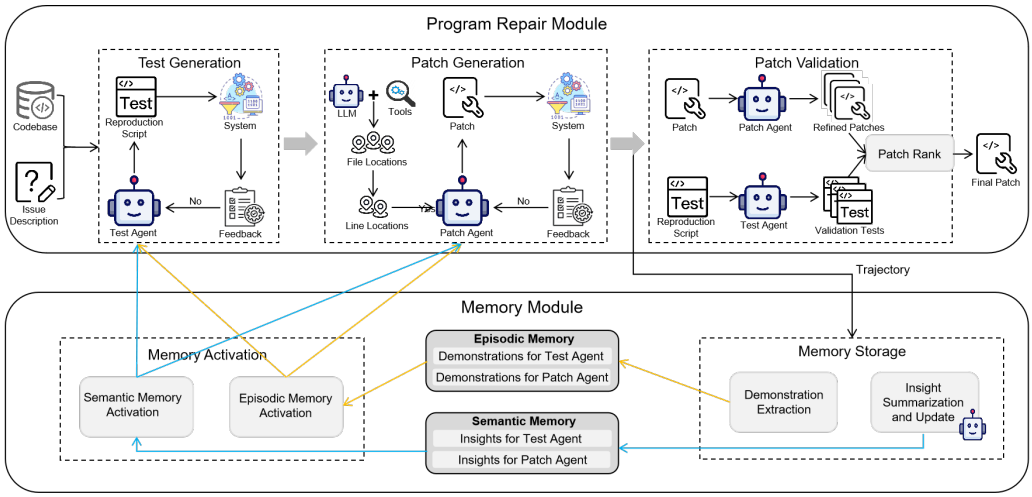


Fig. 1. Overview of ExpeRepair.

prompts remain static. In the inference phase, ExpeRepair activates both memory systems: retrieving relevant demonstrations from episodic memory and recalling reflective insights from semantic memory. This enables ExpeRepair to dynamically compose context-aware, experience-driven prompts, progressively enhancing its repair capabilities.

2.1.1 Initial Phase. In this phase, ExpeRepair is applied to a set of seed issues. To ensure a fair comparison with baseline methods, which are evaluated directly on the SWE-Bench benchmark, we do not incorporate any external data as seed issues. Instead, ExpeRepair follows a straightforward approach: it attempts to reproduce the first five issues from each repository. If an issue is successfully reproduced (as determined by the LLM), ExpeRepair proceeds to perform a complete resolution. The resulting repair trajectories are then stored and organized within the memory module for subsequent recall. Specifically, the LLM agents in the program repair module use the ReAct algorithm [51] to iteratively process their respective tasks up to Z times. Assuming an LLM agent processes the n-th issue at the z-th trial, it is provided with a task prompt P defining its role and task (e.g., generating a patch), issue-specific context C_n (e.g., the issue description or relevant code snippets), and feedback $F_{n,z-1}$ (e.g., execution results) from the $(z-1)$-th trial, which is initialized to the empty (i.e., $F_{n,0} = \emptyset$). The agent then generates an output by sampling: $O_{n,z} \sim \text{LLM}\left(P, C_n, F_{n,z-1}\right)$. After all agents complete their tasks, the memory module records their produced trajectories $\tau_{n,z} = [O_{n,1}, F_{n,1}, \ldots, O_{n,z}, F_{n,z}]$, and subsequently extracts both issue-specific demonstrations and abstract natural language insights for each agent. These are stored in the episodic memory $\mathcal{M}_{\text{epi}}$ and the semantic memory $\mathcal{M}_{\text{sem}}$, which together serve as a springboard for a warm start.

2.1.2 Inference Phase. In this phase, ExpeRepair leverages accumulated experience to guide new issue repairs. Specifically, it first extracts summarized natural language insights I from the semantic memory $\mathcal{M}_{\text{sem}}$, and appends them to the corresponding agent's task prompt P. It then forms a retrieval query $q_{n,z} = [C_n; F_{n,z-1}]$, which concatenates the current issue context with the latest feedback when available, and uses this query to retrieve the top-k relevant demonstrations $D_{n,z} \subset \mathcal{M}_{\text{epi}}$. As a result, for the n-th issue at the z-th trial, each LLM agent is provided with the augmented input to generate the output: $O_{n,z} \sim \text{LLM}\left(P + I, D_{n,z}, C_n, F_{n,z-1}\right)$. Similarly, after all agents complete their tasks, the memory module records the resulting repair trajectories, extracts

new demonstrations and insights, and updates the dual memories. Over time, this process enables ExpeRepair to progressively enrich its episodic and semantic memories, fostering improvement in repair efficiency and effectiveness for future issues.

2.2 Program Repair Module

2.2.1 Test Generation. In practical software development, reproduction tests are essential for both demonstrating the presence of an issue and validating the correctness of candidate patches [32]. However, existing approaches typically prompt an LLM to generate a reproduction test script solely based on the issue description, which presents two major limitations: (1) frequent execution failures due to omitted dependencies, configurations, or environment-specific settings, and (2) inadequate issue reproduction, as such scripts often narrowly target the symptoms described in the issue without capturing the broader fault context.

ExpeRepair addresses these limitations by enabling the test agent to iteratively generate and refine reproduction tests based on dynamic feedback and memories accumulated from past repair trajectories. Specifically, before generating a new reproduction test at trial z for the n-th issue, the test agent retrieves relevant demonstrations and extracts insights from both memories. Concretely, it forms the retrieval query $q_{n,z}^{\text{test}} = [C_n^{\text{test}}; F_{n,z-1}^{\text{test}}]$ and retrieves the top-k demonstrations from episodic memory according to their similarity to this query: $D_{n,z}^{\text{test}} = \arg\text{top-k}_{d \in \mathcal{M}_{\text{epi}}} \text{Sim}\left(q_{n,z}^{\text{test}}, q_d\right)$ where q_d is constructed from the issue context and feedback of each demonstration. This directly addresses the limitation (1): when the current test execution encounters a failure (e.g., a missing library or configuration error), demonstrations associated with the same or similar failures provide concrete examples of how those were successfully handled in the past. Then, the agent extracts all summarized natural language insights I_{test} from the semantic memory $\mathcal{M}_{\text{sem}}$. These insights capture generalizable, high-level strategies distilled from prior repairs. For example: "When testing security-sensitive functionality, implement comprehensive test cases that verify proper handling of malicious inputs, edge cases, and potential attack vectors, ensuring robust validation and appropriate error responses" (a real insight summarized by ExpeRepair for the SymPy project). These insights help address the limitation (2) by expanding the agent's reasoning beyond narrowly reproducing stated symptoms.

After recalling knowledge from the memories, ExpeRepair synthesizes the following adaptive prompt: (1) the enhanced task prompt $P^{\text{test}} + I^{\text{test}}$, (2) the issue description C_n^{test}, (3) the execution feedback $F_{n,z-1}^{\text{test}}$ from the previous test trial, and (4) the retrieved demonstrations $D_{n,z}^{\text{test}}$. Formally, the test agent's output at trial z is generated as: $O_{n,z}^{\text{test}} \sim \text{LLM}\left(P^{\text{test}} + I^{\text{test}}, D_{n,z}^{\text{test}}, C_n^{\text{test}}, F_{n,z-1}^{\text{test}}\right)$. This iterative process continues until either a valid reproduction test is produced or Z^{test} iterations have been completed.

Prompt for Test Generation

Please analyze the issue description to understand the reported problem. Based on your analysis, write a standalone Python script `test.py` to reproduce the issue.
Requirements:
- The script should be minimal, including the essential tests necessary to demonstrate the issue.
- The script must be self-contained and runnable, including all necessary imports, dummy data, and setup.
- Use try-except blocks to isolate and execute each test case safely.
- Print the result of each test case (if applicable) to assist with debugging.

Here are reflective insights summarized from other issues in the same repository:
<summarized_insights>

Below are example test scripts for other issues:
<retrieved_demonstrations>

===###====
<issue-specific context>
<feedback from the last trial>

2.2.2 Patch Generation. Following the successful reproduction of an issue, ExpeRepair initiates patch generation to produce candidate repairs that resolve the fault while preserving existing functionality. Before generating patches, it is essential to localize the issue, as accurate localization is a prerequisite for effective program repair. Following previous work [49], ExpeRepair adopts a hierarchical localization strategy. Specifically, ExpeRepair first extracts the repository structure and provides it, together with the issue description, to the LLM to identify suspicious files. For each file, the LLM is subsequently prompted with the file content and the issue description to pinpoint likely faulty lines. The localization process is fully automated and relies on LLM-based reasoning rather than manually designed heuristics.

As noted in previous studies [6, 49], LLMs often generate incomplete patches, as fixing a repository-level issue typically requires coordinating changes across multiple parts of the code or performing a sequence of modifications. To address it, similar to the test generation process, ExpeRepair iteratively recalls episodic and semantic memories and leverages them to generate and refine patches. At each trial z for the n-th issue, the patch agent retrieves relevant demonstrations and extracts reflective insights, following the same procedure as in the test generation phase. The agent is then prompted with: (1) the enhanced task prompt $P^{\text{patch}} + I^{\text{patch}}$, (2) the issue description and localized faulty lines C_n^{patch}, (3) the latest execution feedback $F_{n,z-1}^{\text{patch}}$, and (4) the retrieved demonstrations $D_{n,z}^{\text{patch}}$. A candidate patch is generated as: $O_{n,z}^{\text{patch}} \sim \text{LLM}\left(P^{\text{patch}} + I^{\text{patch}}, D_{n,z}^{\text{patch}}, C_n^{\text{patch}}, F_{n,z-1}^{\text{patch}}\right)$. In each iteration, we perform multiple samplings with a high temperature to generate diverse candidate patches. This process continues until at least one candidate patch passes the reproduction test (i.e., the issue is no longer triggered) or reaches Z^{patch} iterations.

Prompt for Patch Generation

Your task is to analyze and resolve the given issue in two phases:
Phase 1: FIX ANALYSIS
1. Review the issue description and state clearly what the problem is.
2. Analyze the provided code context and specify where the problem occurs in the code.
3. State clearly the best practices to take into account in the fix.
4. State clearly how to fix the problem.

Phase 2: FIX IMPLEMENTATION
1. Focus on making minimal, precise, and relevant changes to resolve the issue.
2. Include any necessary imports introduced by the patch.
3. Write the patch using the strict format specified below:
<patch_format>

```
Here are reflective insights summarized from other issues in the same repository:
<summarized_insights>

Below are example patches for other issues:
<retrieved_demonstrations>

===###====
<issue-specific context>
<feedback from the last trial>
```

2.2.3 Patch Validation. After a candidate patch successfully passes the reproduction test, it is not immediately accepted as the final patch. This is because we empirically observed that reproduction scripts typically focus narrowly on the specific symptoms described in the issue, leading to false positives where patches pass limited tests but fail in broader contexts. To mitigate this, we revise and augment the patch by instructing the patch agent to address concerns such as edge case handling, regression risks, and adherence to language-specific best practices. Next, we augment the reproduction test suite by generating additional validation tests. The test agent is instructed to create tests targeting boundary conditions and corner cases. This process reduces the risk of narrow, brittle fixes that only address the reported symptom without addressing the underlying defect comprehensively.

To select the final patch, we first execute all candidate patches against the expanded test suite, which includes both reproduction and validation tests. Then, we pass candidate patches and their corresponding test results to the LLM, which selects the final patch based on criteria such as correctness and adherence to best practices.

2.3 Memory Module

2.3.1 Memory Storage. After the program repair module processes a new issue, the memory module collects the complete repair trajectory, which consists of all detailed artifacts associated with that specific repair instance, including the issue description, reproduction scripts, candidate patches, and the test execution results before and after each patch is applied. ExpeRepair begins by extracting issue-specific demonstrations for both the test agent and the patch agent, which are stored in episodic memory. Specifically, for each trajectory, it collects two types of demonstrations: (1) `(input, successful output)` pairs, representing successful repair attempts, and (2) `(input, failed output, feedback, successful output)` tuples, capturing failed attempts along with their outputs, feedback, and corrected versions. The successful/failed output refers to whether the test script successfully reproduces the issue (as determined by the LLM) or whether the patch passes the reproduction test. All collected demonstrations are ultimately stored in episodic memory.

Then, ExpeRepair performs a summarize-and-update process to maintain the semantic memory, which stores high-level natural language insights. We prompt the LLM to analyze the complete repair trajectory and explain why specific attempts to reproduce the issue or generate a patch succeeded or failed. This generates reflective insights that may include general repair strategies, common failure causes, and effective heuristics. The newly generated insights are compared with those already stored in semantic memory, and ExpeRepair updates the memory by applying up to three operations: (1) `ADD`, to introduce a novel or valuable insight; (2) `REMOVE`, to discard contradictory, outdated, or redundant information; and (3) `EDIT`, to merge, generalize, or refine overlapping insights for improved clarity and utility. To maintain a manageable memory size, we impose a maximum number of insight entries per agent. When this limit is reached, ExpeRepair must first `REMOVE` an existing insight before applying a new `ADD`.

Prompt for Insight Summarization and Update

Your task is to:
1. Review the issue, analyze the insights provided by different Testers and Coders, and then summarize the new experiences for Testers and Coders, respectively.
2. Update the existing experiences by adding, editing, and removing them based on new summarized experiences.

To update the existing experiences, you must use the following operations:
ADD: Add a new experience if it is significantly different from existing experiences and could benefit future issues.
REMOVE: Remove an existing experience if it is CONTRADICTORY or SIMILAR/DUPLICATED to other existing experiences.
EDIT: Enhance or rewrite an existing experience to make it more general or valuable. If a new experience is similar to an existing one, merge them into a single improved experience using this operation.

IMPORTANT:
1. Align experiences strictly with the roles:
- For Testers, include only experiences related to writing test cases.
- For Coders, include only experiences related to developing patches, excluding any testing activities.
2. Ensure the new experience is practical, actionable, and broadly applicable to similar issues in the future.
3. Do not remove an existing experience solely because it is irrelevant to the current issue; it may still be useful for future cases. Aim to maintain a diverse set of experiences to help colleagues address a wide range of issues.
4. Perform at most four operations for tester and coder experiences individually, and each existing experience can receive only one operation.
5. The total number of experiences for each role must NOT EXCEED 15.

2.3.2 Memory Activation. To activate episodic memory, ExpeRepair uses the same retrieval query construction described above, namely $q_{n,z} = [C_n; F_{n,z-1}]$, and retrieves the top-$k$ most relevant past demonstrations using retrieval methods such as vector-based similarity (e.g., embedding search [19]) or term-based matching (e.g., BM25 [33]). The retrieved demonstrations are then incorporated into the agent's prompts as in-context examples, offering concrete precedents for constructing accurate reproduction scripts or patches. This memory mechanism enables ExpeRepair to avoid redundant exploration and benefit from accumulated experience, allowing it to generate higher-quality reproduction scripts and patches.

Simultaneously, ExpeRepair activates the semantic memory by extracting all stored insights. The insights are incorporated into the task prompts for both agents to adaptively adjust and enhance their strategies in response to the characteristics of the current issue. This mechanism, effectively serving as a lightweight auto-prompting technique [36, 37, 45], incrementally improves repair effectiveness by guiding agents with accumulated semantic experience.

3 Experimental Setup

3.1 Research Questions

We address the following four research questions to assess the performance of ExpeRepair.

RQ1: How effective is ExpeRepair in issue resolution compared to other approaches?
RQ2: How does each component of ExpeRepair contribute to its overall performance?
RQ3: How does the choice of underlying LLM affect ExpeRepair's performance?
RQ4: How sensitive is ExpeRepair to key hyperparameters in its memory module?

3.2 Benchmark

To evaluate the effectiveness of ExpeRepair in addressing real-world software issues, we conduct experiments on two subsets of the SWE-Bench benchmark [21]: SWE-Bench Lite and SWE-Bench Verified. SWE-Bench Lite consists of 300 GitHub issues sampled from real-world Python projects. SWE-Bench Verified [10], produced by OpenAI, is a curated subset in which each issue has been validated by human developers to ensure it contains sufficient information to be solvable.

3.3 Baselines

Repository-level program repair has received significant attention from both academia and industry. Numerous approaches have been proposed and evaluated on SWE-Bench. In this work, we focus on open-source methods, as the technical details of closed-source systems are generally unavailable. We compare our approach against the following representative open-source baselines:

- **SWE-agent** [49] introduces a general agent–computer interface that enables LLMs to autonomously leverage external tools for fixing issues in real GitHub repositories.
- **Aider** [15] is an LLM-based coding assistant that combines repository-aware context construction with Git-integrated editing to produce context-aware fixes in large codebases.
- **AutoCodeRover** [53] is an autonomous program improvement system that merges LLMs with advanced code search capabilities. It addresses GitHub issues through iterative patch generation and refinement.
- **SpecRover** [35] extends AutoCodeRover by incorporating code intent extraction via LLMs, enhancing issue resolution. On the full SWE-Bench, SpecRover achieves over 50% improvement in efficacy compared to AutoCodeRover, demonstrating its advanced autonomous repair capabilities.
- **Agentless** [46] follows a procedural approach that decomposes the repair process into distinct phases: localization, repair, and patch validation.
- **OpenHands** [42] is a comprehensive framework that leverages LLMs for real-world software engineering tasks. It integrates a sandboxed execution environment, a critic model for ranking solutions, and a dual-agent architecture with experience banks to improve repair performance.
- **PatchPilot** [24] is a procedural repair method that balances efficacy, stability, and cost-efficiency. It guides bug repair through a five-step workflow: reproducing the error, localizing the fault, generating a fix, validating the result, and refining the solution.
- **DARS** [11] introduces an inference-time compute scaling strategy for coding agents. By branching only at critical decisions and providing long-horizon feedback, it reduces resets and accelerates execution, improving overall efficiency.

3.4 Metrics

Following prior work [46, 49, 53], we evaluate the performance of the baselines and ExpeRepair using the following primary metrics:

- **pass@1:** The percentage of issues successfully resolved on the first attempt. This metric reflects the method's ability to generate correct patches without requiring multiple submissions. It represents the most practical scenario for real-world deployment.

- **Average Cost:** The average inference cost incurred when executing the repair method. This metric captures the computational efficiency and resource requirements of each approach, providing a practical measure of its operational overhead.

These primary metrics form the basis for the overall performance comparison in Section 4.1. In addition, we introduce two supplementary metrics in our ablation studies to provide deeper insights into the intermediate stages of the repair process and to better explain the performance differences:

- **Execution Success Rate (ESR):** The percentage of generated reproduction scripts that execute successfully without errors, such as missing dependencies or configuration issues.
- **Reproduction Success Rate (RSR):** The percentage of reproduction scripts that correctly reproduce the target issue, as manually verified by human annotators.

3.5 Implementation

ExpeRepair uses Claude 3.5/4 Sonnet as its primary foundation model throughout the pipeline. o4-mini is invoked only in rare fallback cases (less than 1% of total LLM calls) when temporary API unavailability occurs, following a practice similar to SpecRover [35]. Due to space limitations, we present only the key prompts for test generation, patch generation, and memory storage in Section 2. The remaining prompts are available on our project website [13]. Below, we summarize the implementation details for each phase.

Test and Patch Generation. For each issue, the test agent iteratively generates a reproduction script for up to 3 rounds. Upon successfully reproducing the issue, the patch agent performs up to 3 iterations of patch generation, producing 4 candidate patches per iteration. Failed patches and execution results are fed back to guide new candidates, while passing patches proceed to validation.

Patch Validation. To reduce false positives, ExpeRepair samples 3 validation tests by prompting the LLM to generate edge cases and boundary conditions from the reproduction script. The patch agent then generates 4 augmented patches addressing these cases, regression risks, and language-specific best practices. All candidates are executed against validation tests, and an LLM reviews the results to select the best-performing patch.

Memory Storage. For episodic memory, demonstrations are stored along with metadata (issue description, status, timestamp), as described in Section 2.3.1. Semantic memory stores key insights summarized by the LLM. These insights are compared against existing entries and updated via `ADD`, `REMOVE`, or `EDIT`. The number of stored insights is capped at 15. When this limit is reached, the LLM removes insights deemed outdated or less reliable to make room for new ones.

Memory Activation. For episodic memory, ExpeRepair retrieves the top 3 most relevant demonstrations via the BM25 algorithm, which are then used as in-context examples for the agents. For semantic memory, stored insights are appended to the prompts as high-level repair guidelines, enhancing the agents' reasoning and decision-making capabilities.

4 Results

4.1 Overall Performance

We conduct a comparative evaluation of ExpeRepair against a set of recent open-source baselines on the SWE-Bench Lite and SWE-Bench Verified benchmarks. Note that the first five issues, which are used to initialize the semantic and episodic memory components, are also included in the total issues addressed in the standard sequential order of the evaluation to compute the final resolution rate. This design ensures repair performance is evaluated on all issues and provides a fair comparison with all baselines. Table 1 reports the resolution rate (pass@1) and the average inference cost per instance (USD) for all methods.

Table 1. Comparison of ExpeRepair and open-source baselines on SWE-Bench Lite and Verified benchmarks. For baselines, we directly use the reported results either from the official leaderboard or from the tool's official paper/repository.

Method	LLM	SWE-Bench Lite		SWE-Bench Verified	
		pass@1 (%)	AVG Cost ($)	pass@1 (%)	AVG Cost ($)
SWE-agent	Claude 3.5 Sonnet	23.0	1.62	33.6	1.59
Aider	GPT-4o + Claude 3 Opus	26.3	-	-	-
AutoCodeRover	GPT-4o	30.7	-	-	-
SpecRover	Claude 3.5 Sonnet + GPT-4o	31.0	0.65	46.2	-
Agentless-1.5	Claude 3.5 Sonnet	40.7	1.12	50.8	1.19
OpenHands	CodeAct v2.1	41.7	1.33	53.0	0.78
PatchPilot	Claude 3.5 Sonnet	45.3	0.97	53.6	0.99
DARS	Claude 3.5 Sonnet + DeepSeek R1	47.0	12.24	-	-
ExpeRepair	Claude 3.5 Sonnet + o4-mini	48.3	1.89	57.2	1.74
	Claude 4 Sonnet + o4-mini	60.3	1.96	74.6	1.91

Overall, ExpeRepair achieves state-of-the-art performance across both benchmarks. Using Claude 3.5 Sonnet, ExpeRepair resolves 48.3% of issues on SWE-Bench Lite and 57.2% on SWE-Bench Verified, surpassing all previously published open-source approaches, including PatchPilot [24] and DARS [11]. When paired with Claude 4 Sonnet, ExpeRepair further improves performance, achieving 60.3% resolution on SWE-Bench Lite and 74.6% on SWE-Bench Verified. These results demonstrate both the adaptability of ExpeRepair and the potential benefits of leveraging more powerful underlying LLMs.

In addition to effectiveness, ExpeRepair maintains a competitive inference cost. For example, while DARS achieves a pass@1 of 47.0% on SWE-Bench Lite, its average cost is 12.24 USD per instance, which is roughly six times higher than ExpeRepair's 1.89 USD using Claude 3.5 Sonnet. This highlights that ExpeRepair not only improves repair effectiveness but also offers practical efficiency suitable for real-world deployment. Compared with other strong baselines, ExpeRepair consistently outperforms PatchPilot and OpenHands, which achieve 45.3% and 41.7% resolution on SWE-Bench Lite, respectively, with comparable or slightly higher costs. Similarly, Agentless-1.5 attains 40.7% resolution with an inference cost of 1.12 USD, indicating that while it is cost-efficient, its effectiveness remains substantially lower than that of ExpeRepair.

Answering RQ1: ExpeRepair achieves state-of-the-art performance on both SWE-Bench Lite and SWE-Bench Verified, resolving 48.3% and 57.2% of issues with Claude 3.5 Sonnet, and up to 60.3% and 74.6% with Claude 4 Sonnet. It consistently outperforms all open-source baselines while maintaining competitive inference costs, demonstrating a favorable balance between effectiveness and efficiency.

4.2 Ablation Study

To assess the contribution of individual components in ExpeRepair, we conduct a systematic ablation study on the SWE-Bench Lite and SWE-Bench Verified benchmarks. Unless otherwise specified, all experiments use Claude 3.5 Sonnet as the foundation model.

4.2.1 Ablations on Episodic Memory and Semantic Memory Components. We evaluate the contributions of Episodic and Semantic Memory by disabling them individually or jointly: (1) **w/o Memory Module**, which removes both Episodic and Semantic Memory and treats each issue in isolation, as in prior methods; (2) **w/o Episodic Memory**, which removes the in-context demonstrations

Table 2. Ablations on Episodic Memory and Semantic Memory components

Method	SWE-Bench Lite			SWE-Bench Verified		
	pass@1 (%)	ESR (%)	RSR (%)	pass@1 (%)	ESR (%)	RSR (%)
ExpeRepair	48.3	82.0	78.7	57.2	84.6	82.2
w/o Memory Module	42.3	61.3	57.7	50.4	59.6	57.0
w/o Episodic Memory	44.7	68.0	65.0	52.8	68.4	64.6
w/o Semantic Memory	47.0	77.3	72.3	56.2	78.2	76.6

provided to LLM agents during test and patch generation; and (3) **w/o Semantic Memory**, which removes reflective insights distilled from prior repair experiences.

As shown in Table 2, removing the Memory Module leads to the most significant performance degradation, with the resolution rate dropping from 48.3% to 42.3% on SWE-Bench Lite and from 57.2% to 50.4% on SWE-Bench Verified. The Execution Success Rate (ESR) and Reproduction Success Rate (RSR) also decrease substantially, by 20.7 and 21.0 pp on Lite, and by 25.0 and 25.2 pp on Verified. This underscores the importance of leveraging past repair knowledge to guide LLM agents in reproducing issues and generating correct patches.

Disabling Episodic Memory also degrades performance notably. The pass@1 decreases to 44.7% on SWE-Bench Lite and 52.8% on SWE-Bench Verified, accompanied by ESR and RSR reductions of 14.0/13.7 pp on Lite and 16.2/17.6 pp on Verified. This suggests that the demonstrations stored in the episodic memory provide effective in-context guidance during both test and patch generation. In contrast, removing the Semantic Memory leads to a relatively smaller performance drop. The resolution rates decrease modestly to 47.0% on Lite and 56.2% on Verified, while ESR and RSR drop by 4.7/6.4 pp on Lite and 6.4/5.6 pp on Verified. This indicates that while the insights distilled in semantic memory improve the quality of generated patches, their effect is less pronounced compared to issue-specific demonstrations.

Overall, these results reveal a clear hierarchy in component contributions: the Memory Module has the largest impact, followed by Episodic Memory, and then Semantic Memory. The combination of Episodic and Semantic Memory yields a synergistic effect, producing the highest pass@1, ESR, and RSR metrics across both benchmarks. These findings validate the design rationale of ExpeRepair: leveraging prior experience to provide both in-context demonstrations and reflective insights delivers substantial gains in autonomous program repair.

4.2.2 Ablations on Episodic Memory Retrieval. We evaluate alternative retrieval strategies for Episodic Memory. In addition to the **Term-Based Retrieval** method (BM25) used in ExpeRepair, we evaluate an **Embedding-Based Retrieval** approach that computes cosine similarity between embeddings generated by the Multilingual-E5-Large [41] model. We also consider a **Hybrid Retrieval** strategy that linearly combines BM25 and embedding similarity scores, weighted by α and $1-\alpha$, respectively. We sweep $\alpha \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$ and report the best-performing results.

Figure 2 shows that BM25 achieves the best or tied-best pass@1 on both SWE-Bench Lite (48.3%) and SWE-Bench Verified (57.2%). Embedding-based retrieval underperforms, while the hybrid approach remains competitive but does not surpass BM25. We attribute this to the importance of lexical overlap in failure signals (e.g., error messages, stack traces, and test feedback), which are better captured by term-based matching. Dense retrieval occasionally returns semantically related yet causally different issues. These findings align with prior work [50], suggesting that more complex retrieval does not necessarily improve code generation.

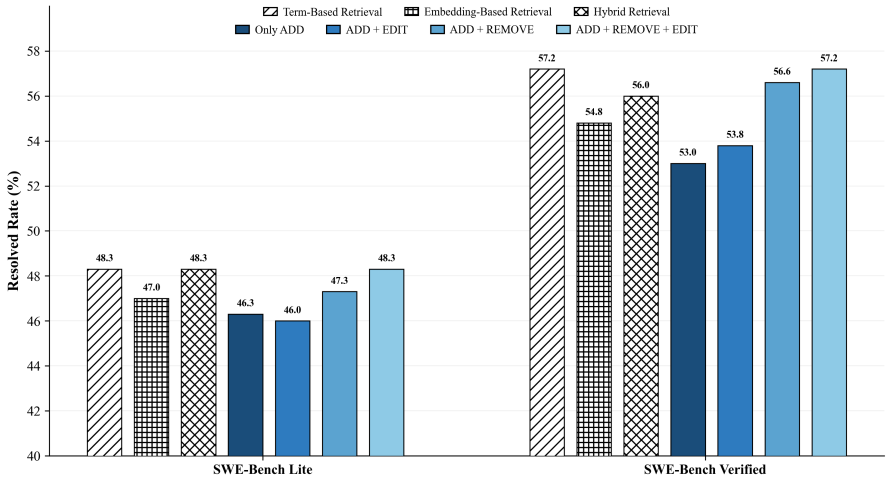


Fig. 2. Ablations on memory retrieval and update strategy.

4.2.3 Ablations on Semantic Memory Update Mechanisms. To further investigate the impact of different Semantic Memory maintenance strategies, we evaluate four variants: (1) **Only ADD**, where new insights are appended without modification or deletion; (2) **ADD+EDIT**, which allows existing entries to be revised but not removed; (3) **ADD+REMOVE**, which supports insertion and deletion without editing; and (4) **ADD+EDIT+REMOVE**, the full strategy used in ExpeRepair. For all variants, when the accumulated semantic insights exceed the model's input limit, we truncate the oldest entries as needed to stay within the input budget.

Figure 2 shows that variants without `REMOVE` consistently underperform, as accumulated insights can saturate the prompt context and introduce noise. Incorporating `REMOVE` yields substantial gains, particularly on SWE-Bench Verified, improving pass@1 by 3.6 pp over `ADD`-only. This underscores the importance of pruning obsolete or misleading entries to maintain memory quality.

While `EDIT` alone provides limited improvement, combining it with `REMOVE` achieves the best performance across both benchmarks. This suggests a complementary effect: `REMOVE` acts as the primary mechanism for controlling memory growth and eliminating noise, `EDIT` further improves memory quality by refining the remaining entries.

Answering RQ2: The ablations confirm that ExpeRepair's gains stem from its dual-memory design. Removing the Memory Module substantially degrades pass@1 on both benchmarks. Episodic Memory contributes the majority of improvements through issue-specific repair demonstrations, while Semantic Memory provides consistent complementary gains via accumulated insights. Further analysis shows that effective memory design requires both accurate retrieval of relevant past experiences and disciplined long-term memory maintenance, with term-based retrieval and incremental memory updates yielding the best overall performance.

4.3 Results on Different LLMs

To evaluate ExpeRepair's performance across different foundation models, we integrate it with four representative models: o1-mini [30], DeepSeek-R1 [12], Claude 3.5 Sonnet [3], and Claude 4 Sonnet [4], while keeping all other components unchanged. As noted in Section 3.5, o4-mini is

invoked only when a query encounters a temporary API error. For brevity, the x-axis labels in Figure 3 list only the different foundation models.

Figure 3 presents the results. Across both benchmarks, we observe a consistent performance ranking among the evaluated models. Claude 4 Sonnet achieves the best performance, followed by Claude 3.5 Sonnet, DeepSeek-R1, and o1-mini. This indicates that ExpeRepair preserves the relative strengths of different models and benefits from stronger underlying models.

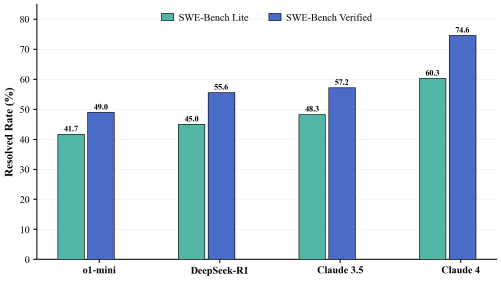


Fig. 3. Results of ExpeRepair on different models.

pass@1 on SWE-Bench Lite ranges from 41.7% (o1-mini) to 60.3% (Claude 4 Sonnet), and from 49.0% to 74.6% on SWE-Bench Verified. Notably, even when paired with relatively less capable models such as o1-mini and DeepSeek-R1, ExpeRepair still achieves competitive results. For example, ExpeRepair with o1-mini attains pass@1 performance comparable to SpecRover built on the Claude 3.5 Sonnet model.

Answering RQ3: The experiments demonstrate that ExpeRepair is broadly compatible with diverse LLM backbones across both benchmarks. Furthermore, stronger base models consistently achieve higher performance, underscoring the importance of model selection when deploying automated code repair systems in practice.

4.4 Sensitivity Analysis of Hyperparameters

In this section, we analyze the sensitivity of ExpeRepair to two key hyperparameters: (1) the number of demonstrations retrieved from episodic memory, and (2) the number of insights stored in semantic memory. Each parameter is varied independently while keeping all other settings fixed. All experiments are conducted on SWE-Bench Lite and SWE-Bench Verified using Claude 3.5 Sonnet as the backbone model.

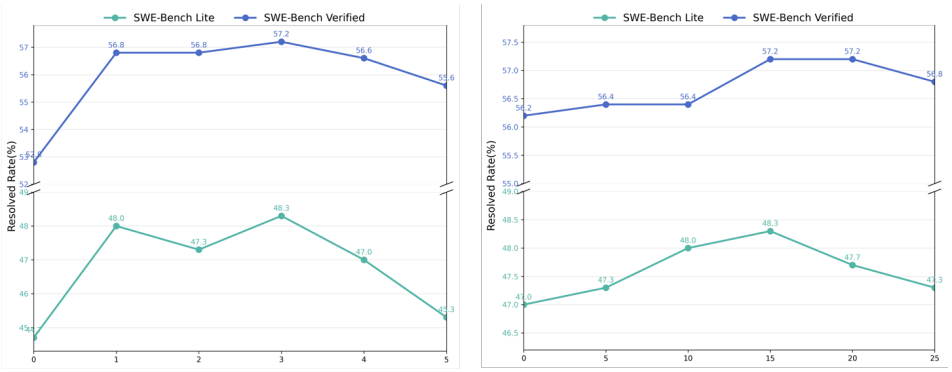


Fig. 4. Sensitivity analysis of the number of retrieved demonstrations and stored insights.

4.4.1 Number of Retrieved Demonstrations. ExpeRepair retrieves the top-k most relevant demonstrations to enhance the agent's reasoning process. We vary k from 0 (no demonstrations) to 5. The results are shown in Figure 4(a).

Across both benchmarks, introducing at least one retrieved demonstration leads to substantial improvements over $k = 0$. Specifically, performance increases from 44.7% to 48.0% on Lite and from 52.8% to 56.8% on Verified when increasing k from 0 to 1. This confirms the important role of episodic memory in improving program repair performance through leveraging prior task examples.

Performance peaks at $k = 3$ on both datasets, reaching 48.3% on Lite and 57.2% on Verified. Further increasing k degrades results (45.3% and 55.6% at $k = 5$), suggesting a trade-off between retrieval breadth and contextual interference. While retrieving more demonstrations increases the likelihood of including relevant examples, it also raises the risk of incorporating less relevant content and consuming additional context budget, which may dilute attention to the current issue.

4.4.2 Number of Stored Insights. We vary the number of stored insights from 0 to 25 in increments of 5, as shown in Figure 4(b).

On SWE-Bench Lite, performance improves from 47.0% (no insights) to a peak of 48.3% at 15 insights, followed by a gradual decline to 47.3% at 25 insights. A similar pattern is observed on Verified, where performance increases from 56.2% to 57.2% at 15–20 insights, before slightly decreasing to 56.8% at 25 insights.

These results indicate that semantic memory is beneficial up to a moderate size (approximately 15–20 insights), after which gains plateau or slightly diminish, likely due to redundancy or lower-utility information within the constrained context window.

Answering RQ4: ExpeRepair demonstrates stable and robust behavior across a reasonable range of hyperparameter settings. The empirically best configuration, i.e., 15 stored insights and top-3 demonstration retrieval, achieves 48.3% on Lite and 57.2% on Verified. Importantly, performance remains near-optimal within moderate deviations (10–20 insights and $k = 2$–4), indicating that the method is not overly sensitive to minor hyperparameter variations.

5 Discussion

5.1 Case Study

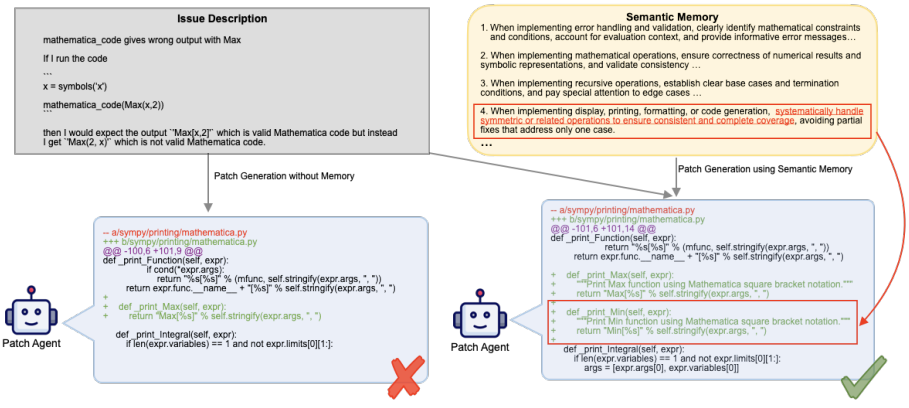
To further evaluate ExpeRepair's effectiveness, we present two real-world cases showing how semantic and episodic memory improve agents' decisions in bug reproduction and patch generation.

5.1.1 Semantic Memory. Figure 5 (a) illustrates an issue in `SymPy` related to incorrect code generation for the `Max` function when using the `mathematica_code` printer. Specifically, given the input

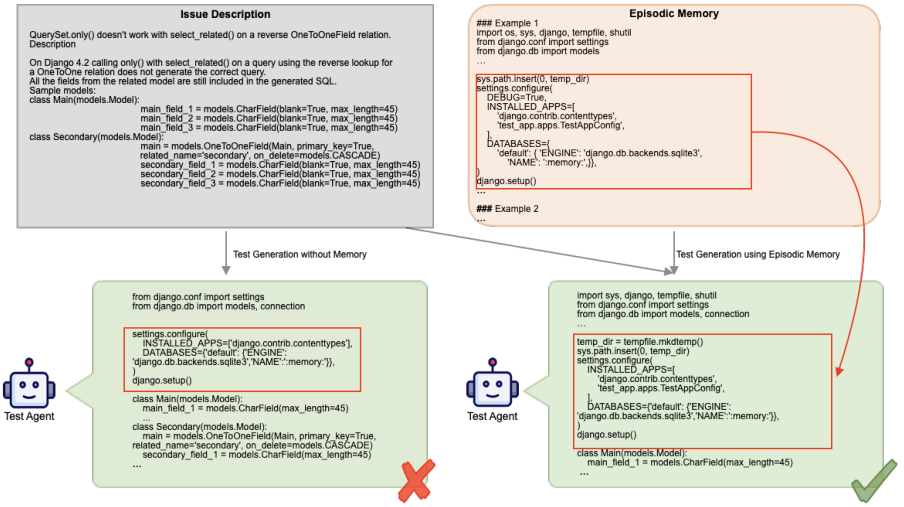
```
x = symbols('x');  mathematica_code(Max(x,2))
```

the expected output is `Max[x,2]`, which conforms to valid Mathematica syntax. Instead, the current implementation produces `Max(2, x)`, which violates Mathematica's function-call conventions. This behavior arises because the `mathematica_code` printer mishandles variadic functions such as `Max`, generating syntactically invalid code. Although the issue is framed in terms of `Max`, the same underlying problem affects its counterpart function `Min`. This highlights the need for solutions that systematically handle symmetric or related operations to ensure consistent code generation.

Initially, the patch agent attempted to resolve the issue without activating semantic memory. The generated patch introduced a specialized `_print_Max` method to enforce Mathematica's square-bracket notation. While this patch correctly formats `Max`, it fails to address the counterpart function



(a) Case study of ExpeRepair on instance SymPy-15345



(b) Case study of ExpeRepair on instance Django-16910

Fig. 5. Two real cases from the SymPy and Django repositories resolved by ExpeRepair.

`Min`, leaving it incorrectly printed. Consequently, although `Max` expressions are valid, the patch produces asymmetric and incomplete code generation.

Subsequently, the patch agent was allowed to activate semantic memory, incorporating all previously stored reflective insights into its system prompt. Notably, the fourth insight, *"When implementing display, printing, formatting, or code generation, systematically handle symmetric or related operations to ensure consistent and complete coverage, avoiding partial fixes that address only one case."*, which was derived from resolving similar issues such as SymPy-13031, guides the agent to consider not only the explicitly reported `Max` function but also `Min` and other symmetric operators. As a result, the newly generated patch resolves the reported `Max` defect and simultaneously addresses `Min`, yielding a robust and comprehensive solution. This ensures consistent and syntactically valid Mathematica code generation across all relevant variadic functions, effectively preventing asymmetric or incomplete code generation.

5.1.2 Episodic Memory. As shown in Figure 5(b), this case examines a regression in Django 4.2 concerning the interaction between `QuerySet.only()` and `select_related()` when applied to

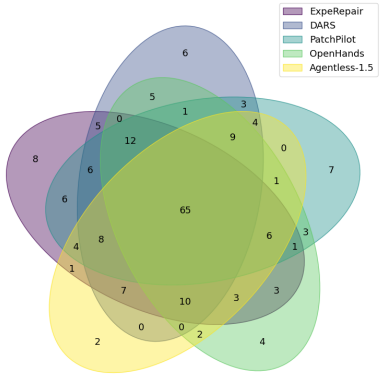


Fig. 6. Intersection analysis.

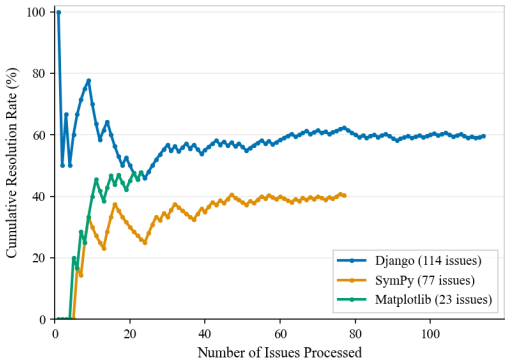


Fig. 7. Cumulative resolution rates across repositories.

reverse `OneToOneField` relationships. In earlier versions (e.g., Django 4.1), restricting fields with `only()` correctly limited both the primary model and its related reverse model to the specified subset of fields. However, in Django 4.2, combining `only()` with `select_related()` on a reverse `OneToOneField` fails to propagate field restrictions to the related model, resulting in all fields of the related model being included in the generated SQL. This deviation from expected behavior leads to unnecessary data retrieval, potentially impacting performance and memory usage.

In the initial attempt, the test agent tried to reproduce the issue without leveraging episodic memory. The generated script was ineffective due to an incomplete test setup: it did not establish a full Django application with proper `INSTALLED_APPS`, lacked complete database table creation and test data population, and did not force query execution to generate SQL. Consequently, the queries either did not execute or did not reflect the field restrictions necessary to trigger the regression.

In a subsequent attempt, the agent accessed episodic memory to retrieve prior examples of testing scripts. The retrieved example correctly sets up a temporary Django application, creates the relevant models and test data, and executes queries in a controlled environment. Guided by these examples, the agent produced a fully functional test script that successfully reproduced the issue: when combining `select_related()` with `only()`, all fields of the reverse `OneToOneField` model were included in the generated SQL, confirming the regression in Django 4.2.

5.2 Intersection Analysis

We further conduct an intersection analysis comparing the issues resolved by ExpeRepair against four leading baselines. Figure 6 presents the complete overlap structure on SWE-Bench Lite.

As shown in Figure 6, ExpeRepair uniquely resolves 8 issues that no other open-source method fixes. Meanwhile, DARS and PatchPilot uniquely solve 6 and 7 issues, respectively, indicating that the repair capabilities of different methods are partially complementary rather than dominated by a single approach. In addition, 65 issues are solved by all methods, suggesting a shared ability to address relatively straightforward bugs.

We manually inspected the 8 issues uniquely fixed by ExpeRepair and found that six of their solutions follow recurring repair patterns derived from accumulated experience, which include: (i) 'add missing validation', (ii) 'generalize single-value to collection', and (iii) 'correct attribute or type resolution'. For example, in `django__django-15320`, `Subquery.as_sql()` produced invalid SQL because the query's `subquery` flag was unset. ExpeRepair correctly set `self.query.subquery = True` in the Subquery constructor by transferring the pattern of ensuring proper initialization from previous fixes, guiding the patch generation process.

At the same time, issues solved exclusively by DARS highlight a limitation of ExpeRepair: its effectiveness drops when relevant prior experience is unavailable. Four of the six DARS-only issues require novel solutions with few analogues in the repository, such as introducing new classes or overrides with custom behavior. Such cases favor DARS, whose structured branching and iterative refinement are better suited to exploratory reasoning than to analogy-based retrieval.

These findings suggest that ExpeRepair and the SOTA baselines exhibit complementary inductive biases, and that hybrid approaches leveraging their respective strengths offer a promising direction for future research.

5.3 Longitudinal Study

To assess whether ExpeRepair's accumulated memory enhances repair performance, we conduct a longitudinal study to track how the cumulative resolution rate evolves as issues are processed. Since ExpeRepair maintains separate episodic and semantic memories per repository, we selected the three repositories in SWE-Bench Lite with the largest issue counts to ensure sufficient temporal depth for observing potential learning effects. Issues were processed in the chronological order provided by the dataset, and after each issue, we computed the cumulative resolution rate as the proportion of resolved issues among all issues processed up to that point.

As shown in Figure 7, all repositories exhibit high volatility during the early phase. Django's cumulative resolution rate fluctuates between 50% and 100% during the first 20 issues, whereas the resolution rates of SymPy and Matplotlib rise from zero with pronounced swings. After approximately 20 issues, the cumulative resolution rates begin to stabilize and gradually improve, suggesting that newly accumulated memory enhances coverage of recurring issue patterns, thereby improving overall performance and reducing variance. Django's cumulative resolution rate stabilizes around issue 80 at approximately 60%, which may be attributable to diminishing returns from repeated memory and the increasing heterogeneity of the remaining issues.

5.4 Limitation

While our approach improves test generation and patch generation by leveraging accumulated episodic and semantic memories, it currently places less emphasis on optimizing bug localization, which is also a critical factor in program repair performance. The primary challenge lies in the difficulty of verifying localization correctness. Unlike test generation and patch generation, whose correctness can be directly validated through execution outcomes (e.g., triggering issues or passing test cases), bug localization lacks an automated oracle. The correctness of a localization prediction cannot be reliably determined based on the success or failure of the subsequent patch. One possible solution is to selectively accumulate localization experiences only from successful repair attempts, assuming that a correct patch implies the localization was correct. However, this conservative strategy would result in memories containing only a narrow subset of successful localization cases, limiting both diversity and coverage. We leave the integration of reliable, memory-based bug localization into our framework as an important direction for future work.

5.5 Threats to Validity

The first threat to validity is the potential for data leakage. Since LLMs are often trained on open-source repositories, public benchmarks may inadvertently overlap with their training corpora. This raises the risk that evaluation results reflect prior exposure rather than genuine generalization. To mitigate this concern, we avoid long-standing APR benchmarks (e.g., Defects4J [22]), which are more likely to have been included in model training corpora. Instead, we adopt SWE-Bench, a more recent benchmark that is less susceptible to contamination and has gained broad adoption in recent studies [11, 46, 47]. Moreover, our evaluation emphasizes the reasoning process of models

rather than their ability to recall existing solutions. Across experiments, our approach consistently outperforms strong baselines built on the same underlying models, suggesting that the observed gains stem from our methodology rather than memorization. In future work, we plan to extend our assessment to contamination-free datasets, such as live-updatable benchmarks [52], to further strengthen the robustness of our findings.

A second threat arises from the limited scope of our experiments, primarily constrained by computational resources. Although we evaluate four models from three different providers, our study still covers only a limited set of representative LLMs rather than the full spectrum of open- and closed-source architectures. This may introduce model-selection bias and limit the generalizability of our findings. To mitigate this concern, we draw on prior work in repository-level APR [46], which has shown the validity of evaluation on a limited number of models. Moreover, the consistent improvements observed across the evaluated models indicate that our approach is not tied to a particular model and is adaptable to diverse architectures.

The third threat concerns the generalizability of our approach beyond Python codebases. Our evaluation is limited to open-source projects implemented in Python, constraining our ability to demonstrate cross-language effectiveness. However, ExpeRepair is designed to be language-agnostic. It leverages experiential knowledge from test generation and bug repair patterns, rather than relying on language-specific syntax or semantics. This design suggests that our approach can extend to other programming languages, such as C# and Ruby.

6 Related Work

6.1 Memory-Enhanced LLM Agents

To enhance LLM agents' ability to retain and exploit prior experiences, researchers have proposed a variety of memory-augmented architectures [26, 48, 54, 55].

Early explorations focused on human-inspired memory systems for long-term interactions [26, 55]. For instance, Think-in-Memory (TiM) [26] introduced a two-phase paradigm consisting of pre-generation recall and post-generation memory refinement, allowing LLMs to evolve their memory representations without redundant reasoning steps. MemoryBank [55] distinguished between short- and long-term memory to enable more natural human–AI communication. MemGPT [31] adopted a hierarchical memory organization for extended context management. Collectively, these pioneering efforts highlighted the necessity of persistent memory systems, although their primary emphasis remained on dialogue continuity rather than specialized problem-solving tasks.

More recent frameworks move beyond conversational memory and aim to capture procedural knowledge derived from agent–environment interactions [9, 48, 54]. For example, ExpeL [54] extracts natural language insights and manages them via weighted voting (ADD, EDIT, UPVOTE, DOWNVOTE) for non-parametric learning, while AgentRR [14] adopts a record-and-replay mechanism that logs environmental dynamics and decision trajectories. Building on these efforts, A-Mem [48] introduces a Zettelkasten-inspired architecture with continuously linked notes for scalable retrieval, and Mem0 [9] extracts key facts via a lightweight two-stage process, with its graph-based extension Mem0^g supporting temporal and multi-hop reasoning. Complementing these, AriGraph [2] unifies semantic and episodic memory within a dynamic knowledge graph world model, allowing agents to reason over both abstract knowledge and concrete experiences, thereby enhancing planning and exploration in interactive environments.

The findings of previous work motivate the study presented in this paper. Our work differs from prior research in that it focuses on addressing the unique challenges of large-scale, collaborative software maintenance, bridging the gap between autonomous agent reasoning and practical program repair in real-world repositories.

6.2 Repository-Level Automatic Program Repair

Repository-level APR has attracted increasing attention from both academia and industry [6–8, 46, 49, 53], motivated by the growing complexity of managing bug reports and issue tracking in collaborative platforms such as GitHub. With the rapid progress of LLMs in generative software engineering, LLM-based solutions have become the dominant paradigm for this problem space.

Early methods primarily relied on Retrieval-Augmented Generation (RAG) [29], which retrieves relevant code snippets to guide LLMs during patch generation [21]. More recently, agent-based frameworks treat LLMs as planning agents that iteratively interact with external tools to perform end-to-end issue resolution. Representative examples include SWE-agent [49], which introduced a general agent–computer interface, together with Moatless [56], OpenHands [42], AutoCodeRover [53], and SpecRover [35], which extend this paradigm with improved localization, multi-agent collaboration, and tighter tooling integration. SWE-Search [5] and DARS [1] further enhance agent capabilities through better search and exploration strategies.

In parallel, procedural methods follow expert-designed, fixed workflows that decompose repair into structured phases (e.g., reproduction, localization, patching, validation), as exemplified by Agentless [46] and PatchPilot [24]. Training-based pipelines such as SWE-Fixer [47] and MCTS-Refined CoT [43] further improve performance via continued training on real-world or synthetic repair data.

Although existing research has achieved promising results in repository-level APR, current approaches face a fundamental limitation: they typically treat each issue in isolation and lack systematic mechanisms to leverage historical repair experiences. To address this gap, ExpeRepair introduces a dual-memory system that continuously accumulates, organizes, and reuses past repair trajectories. By enabling dynamic prompt adaptation during inference, ExpeRepair improves repair effectiveness without relying on costly fine-tuning or complex agent orchestration.

7 Conclusion

In real-world software development, developers continually accumulate knowledge from past debugging and repair activities, refining their skills and strategies over time. Inspired by this process, we present ExpeRepair, a novel LLM-based APR method that continuously accumulates and reuses historical repair experiences via a dual-memory system. By integrating episodic and semantic memories, ExpeRepair enables dynamic, context-aware prompt adaptation, enhancing both repair effectiveness and efficiency. We evaluate ExpeRepair on the SWE-Bench Lite and SWE-Bench Verified benchmarks, achieving 60.3% pass@1 on SWE-Bench Lite and 74.6% pass@1 on SWE-Bench Verified using Claude 4 Sonnet. These results demonstrate that leveraging past repair knowledge significantly improves performance on challenging program repair tasks.

8 Data Availability

The data and code necessary to replicate the results of this paper are publicly available at https://github.com/ExpeRepair/ExpeRepair.

Acknowledgments

We sincerely appreciate anonymous reviewers for their constructive and insightful suggestions for improving this manuscript. This work was supported by the National Natural Science Foundation of China Grant No.62232016, Key Project of ISCAS (Grant no. ISCAS-ZD-202401), Basic Research Program of ISCAS (Grant no. ISCAS-JCZD-202304), Innovation Team 2024 ISCAS (No. 2024-66).

References

[1] Vaibhav Aggarwal, Ojasv Kamal, Abhinav Japesh, Zhijing Jin, and Bernhard Schölkopf. 2025. DARS: Dynamic Action Re-Sampling to Enhance Coding Agent Performance by Adaptive Tree Traversal. *CoRR* abs/2503.14269 (2025). arXiv:2503.14269 doi:10.48550/ARXIV.2503.14269

[2] Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Andrey Kravchenko, Mikhail Burtsev, and Evgeny Burnaev. 2024. Arigraph: Learning knowledge graph world models with episodic memory for llm agents. *arXiv preprint arXiv:2407.04363* (2024).

[3] anthropic. 2025. Claude 3.5 Sonnet V2. https://www.anthropic.com/news/claude-3-5-sonnet.

[4] anthropic. 2025. Claude 4 Sonnet. https://www.anthropic.com/claude/sonnet.

[5] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Yang Wang. 2025. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. https://openreview.net/forum?id=G7sIFXugTX

[6] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).

[7] Yuxiao Chen, Jingzheng Wu, Xiang Ling, Changjiang Li, Zhiqing Rui, Tianyue Luo, and Yanjun Wu. 2024. When large language models confront repository-level automatic program repair: How well they done?. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 459–471.

[8] Yuxiao Chen, Jingzheng Wu, Xiang Ling, Changjiang Li, Zhiqing Rui, Tianyue Luo, and Yanjun Wu. 2024. When Large Language Models Confront Repository-Level Automatic Program Repair: How Well They Done?. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE Companion 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 459–471. doi:10.1145/3639478.3647633

[9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *CoRR* abs/2504.19413 (2025). arXiv:2504.19413 doi:10.48550/ARXIV.2504.19413

[10] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Kevin Liu, and Aleksander Madry. 2024. Introducing SWE-bench Verified. OpenAI Blog (2024). https://openai.com/index/introducing-swe-bench-verified/.

[11] DARS. 2025. DARS. https://github.com/vaibhavagg303/DARS-Agent?tab=readme-ov-file.

[12] DeepSeek. 2025. DeepSeek-R1. https://github.com/deepseek-ai/DeepSeek-R1.

[13] ExpeRepair. 2025. ExpeRepair project. https://github.com/ExpeRepair/ExpeRepair.

[14] Erhu Feng, Wenbo Zhou, Zibin Liu, Le Chen, Yunpeng Dong, Cheng Zhang, Yisheng Zhao, Dong Du, Zhi-Hua Zhou, Yubin Xia, and Haibo Chen. 2025. Get Experience from Practice: LLM Agents with Record & Replay. *CoRR* abs/2505.17716 (2025). arXiv:2505.17716 doi:10.48550/ARXIV.2505.17716

[15] Paul Gauthier. 2024. Aider is AI pair programming in your terminal. https://aider.chat/.

[16] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2018. Automatic software repair: A survey. In *Proceedings of the 40th International Conference on Software Engineering*. 1219–1219.

[17] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *CoRR* abs/2401.14196 (2024). arXiv:2401.14196 doi:10.48550/ARXIV.2401.14196

[18] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680* (2024).

[19] Gísli R. Hjaltason and Hanan Samet. 2003. Properties of Embedding Methods for Similarity Searching in Metric Spaces. *IEEE Trans. Pattern Anal. Mach. Intell.* 25, 5 (2003), 530–549. doi:10.1109/TPAMI.2003.1195989

[20] Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, and Siheng Chen. 2025. Self-Evolving Multi-Agent Collaboration Networks for Software Development. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. https://openreview.net/forum?id=4R71pdPBZp

[21] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).

[22] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *International Symposium on Software Testing and Analysis, ISSTA '14, San Jose, CA, USA - July 21 - 26, 2014*, Corina S. Pasareanu and Darko Marinov (Eds.). ACM, 437–440. doi:10.1145/2610384.2628055

[23] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65.

[24] Hongwei Li, Yuheng Tang, Shiqi Wang, and Wenbo Guo. 2025. PatchPilot: A Stable and Cost-Efficient Agentic Patching Framework. *arXiv preprint arXiv:2502.02747* (2025).

[25] Yalan Lin, Yingwei Ma, Rongyu Cao, Binhua Li, Fei Huang, Xiaodong Gu, and Yongbin Li. 2024. LLMs as Continuous Learners: Improving the Reproduction of Defective Code in Software Issues. *CoRR* abs/2411.13941 (2024). arXiv:2411.13941 doi:10.48550/ARXIV.2411.13941

[26] Lei Liu, Xiaoyan Yang, Yue Shen, Binbin Hu, Zhiqiang Zhang, Jinjie Gu, and Guannan Zhang. 2023. Think-in-Memory: Recalling and Post-thinking Enable LLMs with Long-Term Memory. *CoRR* abs/2311.08719 (2023). arXiv:2311.08719 doi:10.48550/ARXIV.2311.08719

[27] Benjamin Livshits and Thomas Zimmermann. 2005. Dynamine: finding common error patterns by mining software revision histories. *ACM SIGSOFT Software Engineering Notes* 30, 5 (2005), 296–305.

[28] Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. 2024. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. *arXiv preprint arXiv:2406.01422* (2024).

[29] Elijah Mansur, Johnson Chen, Muhammad Anas Raza, and Mohammad Wardat. 2024. RAGFix: Enhancing LLM Code Repair Using RAG and Stack Overflow Posts. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 7491–7496.

[30] OpenAI. 2025. OpenAI o1-mini. https://openai.com/index/openai-o1-mini-advancing-cost-efficient-reasoning/.

[31] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *CoRR* abs/2310.08560 (2023). arXiv:2310.08560 doi:10.48550/ARXIV.2310.08560

[32] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. 2015. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 2015 international symposium on software testing and analysis*. 24–36.

[33] Stephen E. Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Found. Trends Inf. Retr.* 3, 4 (2009), 333–389. doi:10.1561/1500000019

[34] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open Foundation Models for Code. *CoRR* abs/2308.12950 (2023). arXiv:2308.12950 doi:10.48550/ARXIV.2308.12950

[35] Haifeng Ruan, Yuntong Zhang, and Abhik Roychoudhury. 2025. SpecRover: Code Intent Extraction via LLMs. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 963–974. doi:10.1109/ICSE55347.2025.00080

[36] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. *CoRR* abs/2402.07927 (2024). arXiv:2402.07927 doi:10.48550/ARXIV.2402.07927

[37] Taylor Shin, Yasaman Razeghi, Robert L. Logan IV, Eric Wallace, and Sameer Singh. 2020. AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, 4222–4235. doi:10.18653/V1/2020.EMNLP-MAIN.346

[38] André Silva, Sen Fang, and Martin Monperrus. 2023. Repairllama: Efficient representations and fine-tuned adapters for program repair. *arXiv preprint arXiv:2312.15698* (2023).

[39] Endel Tulving. 1972. Episodic and Semantic Memory. In *Organization of Memory*, Endel Tulving and Wayne Donaldson (Eds.). Academic Press, New York, 381–403.

[40] Endel Tulving. 1985. How Many Memory Systems Are There? *American Psychologist* 40, 4 (1985), 385–398. doi:10.1037/0003-066X.40.4.385

[41] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2024. Multilingual E5 Text Embeddings: A Technical Report. *arXiv preprint arXiv:2402.05672* (2024).

[42] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. Openhands: An open platform for ai software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*.

[43] Yibo Wang, Zhihao Peng, Ying Wang, Zhao Wei, Hai Yu, and Zhiliang Zhu. 2025. MCTS-Refined CoT: High-Quality Fine-Tuning Data for LLM-Based Repository Issue Resolution. *CoRR* abs/2506.12728 (2025). arXiv:2506.12728 doi:10.48550/ARXIV.2506.12728

[44] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari A. Sahraoui. 2023. Exploring Parameter-Efficient Fine-Tuning Techniques for Code Generation with Large Language Models. *CoRR* abs/2308.10462 (2023). arXiv:2308.10462 doi:10.48550/ARXIV.2308.10462

[45] Yurong Wu, Yan Gao, Bin Zhu, Zineng Zhou, Xiaodi Sun, Sheng Yang, Jian-Guang Lou, Zhiming Ding, and Linjun Yang. 2024. StraGo: Harnessing Strategic Guidance for Prompt Optimization. In *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, 10043–10061. https://aclanthology.org/2024.findings-emnlp.588

[46] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).

[47] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 1123–1139. https://aclanthology.org/2025.findings-acl.62/

[48] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. *CoRR* abs/2502.12110 (2025). arXiv:2502.12110 doi:10.48550/ARXIV.2502.12110

[49] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[50] Zezhou Yang, Sirong Chen, Cuiyun Gao, Zhenhao Li, Xing Hu, Kui Liu, and Xin Xia. 2025. An Empirical Study of Retrieval-Augmented Code Generation: Challenges and Opportunities. *ACM Trans. Softw. Eng. Methodol.* 34, 7 (2025), 188:1–188:28. doi:10.1145/3717061

[51] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

[52] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Yudong Zhang. 2025. SWE-bench Goes Live! *CoRR* abs/2505.23419 (2025). arXiv:2505.23419 doi:10.48550/ARXIV.2505.23419

[53] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. arXiv:2404.05427 [cs.SE] https://arxiv.org/abs/2404.05427

[54] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (Eds.). AAAI Press, 19632–19642. doi:10.1609/AAAI.V38I17.29936

[55] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (Eds.). AAAI Press, 19724–19731. doi:10.1609/AAAI.V38I17.29946

[56] Albert Örwall. 2024. Moatless Tools. https://github.com/aorwall/moatless-tools.

Received 2025-09-12; accepted 2026-03-24